Introduction to Trees in Data Structures

A **Tree** is a **non-linear data structure** that stores data in a **hierarchical** form (like a family tree or folder structure).

It consists of **nodes** connected by **edges**.

Important Terms in Trees

- **Node** → Represents a value
- **Root** → The topmost node
- **Parent** → Node that has children
- **Child** → Node coming from a parent
- **Leaf Node** → Node with no children
- **Internal Node** → Node that has at least one child
- **Edge** → Connection between two nodes
- **Height** → Longest path from root to a leaf
- **Depth** → Distance from root to a node
- **Subtree** → Any child node and its descendants

Why Use Trees?

- Fast searching and sorting
- Used in databases, file systems, compilers
- Represent hierarchical data
- Efficient insertion and deletion

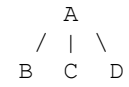
Types of Trees in DSA

Below are the major types of trees you must know:

General Tree

A tree where a node can have **any number of children**.

Example:



Used in: File systems, folders structure

Binary Tree

A tree where **each node has at most 2 children**
(Left child and Right child)



```
 5    20
```

Types of Binary Trees

1. **Full Binary Tree**
   Each node has **0 or 2 children**.
2. **Complete Binary Tree**
   All levels are completely filled except last.
3. **Perfect Binary Tree**
   All interior nodes have 2 children and all leaves are at same level.

Binary Search Tree (BST)

A binary tree with special properties:

- Left subtree contains **smaller** values
- Right subtree contains **greater** values
- No duplicates

```
   40
  /  \
20    60
```

Fast search: **O(log n)**

AVL Tree

AVL (Adelson-Velsky and Landis) tree is a **self-balancing BST**.

Properties:

- Height difference between left & right subtree = **≤ 1**
- Balances using **rotations**

Time Complexity: O(log n)

Red-Black Tree

A self-balancing BST used in STL (C++ map, set).

Properties:

- Every node is **red or black**
- Maintains balanced height
- Root is always **black**

Used in:

- `map`, `set`, `multimap`, `multiset`

Heap Tree

A **complete binary tree** used mainly for priority queues.

Two types:

- **Min-Heap** → Parent is smaller than children
- **Max-Heap** → Parent is larger than children

Used in:

- Priority Queue
- Heap Sort
- Scheduling algorithms

Binary Trees & Binary Tree Traversals

Binary Tree

A **Binary Tree** is a hierarchical data structure where:

- Each node has **at most 2 children**
- Children are called:
  - **Left child**
  - **Right child**

It is used in:

- Expression trees
- Searching
- Sorting
- Hierarchical data representation
- Implementing Binary Search Trees (BST), Heaps, AVL Tree etc.

Basic Terminology

Term	Meaning
Node	Element of the tree (contains data + links)
Root	First/top node of the tree
Parent	Node that has children
Child	Descendant of a parent node
Leaf Node	Node with no children
Subtree	Tree formed from any node
Height	Longest path from root to leaf
Depth	Distance of a node from the root

Binary Tree Node Structure in C++

```
#include <iostream>
using namespace std;

class Node {
public:
    int data;
    Node* left;
    Node* right;

    Node(int value) {
        data = value;
        left = right = nullptr;
    }
};
```

Insertion (User-defined or manual tree building)

Binary Trees do not follow a fixed rule like BST.
We generally insert manually or using level-order.

Example: Manual insertion in left/right:

```
Node* insertLeft(Node* root, int value) {
    root->left = new Node(value);
    return root->left;
}

Node* insertRight(Node* root, int value) {
    root->right = new Node(value);
    return root->right;
}
```

Traversals (MOST IMPORTANT)

Tree traversals are ways to visit every node.

A. Inorder Traversal (Left → Root → Right)

```
L  R  R
```

Example Output for BST → Sorted order.

C++ Code:

```
void inorder(Node* root) {
    if (root == nullptr) return;

    inorder(root->left);
    cout << root->data << " ";
    inorder(root->right);
}
```

B. Preorder Traversal (Root → Left → Right)

Used to create copy of tree.

```
void preorder(Node* root) {
    if (root == nullptr) return;

    cout << root->data << " ";
    preorder(root->left);
    preorder(root->right);
}
```

C. Postorder Traversal (Left → Right → Root)

Used to delete tree bottom-up.

```
void postorder(Node* root) {
    if (root == nullptr) return;

    postorder(root->left);
    postorder(root->right);
    cout << root->data << " ";
}
```

Search in Binary Tree

(General Binary Tree → no order)

```
bool search(Node* root, int key) {
    if (root == nullptr) return false;
    if (root->data == key) return true;

    return search(root->left, key) || search(root->right, key);
}
```

Height of a Binary Tree

Height = max depth from root.

```
int height(Node* root) {
    if (root == nullptr) return 0;

    return 1 + max(height(root->left), height(root->right));
}
```

Count Total Nodes

```
int countNodes(Node* root) {
    if (root == nullptr) return 0;
    return 1 + countNodes(root->left) + countNodes(root->right);
}
```

Count Leaf Nodes

```
int countLeaf(Node* root) {
    if (root == nullptr) return 0;
    if (root->left == nullptr && root->right == nullptr) return 1;

    return countLeaf(root->left) + countLeaf(root->right);
}
```

Level Order Traversal (Breadth First Search)

Use **queue**.

```
#include <queue>

void levelOrder(Node* root) {
    if (root == nullptr) return;

    queue<Node*> q;
    q.push(root);

    while (!q.empty()) {
        Node* curr = q.front();
        q.pop();

        cout << curr->data << " ";

        if (curr->left)  q.push(curr->left);
        if (curr->right) q.push(curr->right);
    }
}
```

Complete Example Program

```
#include <iostream>
#include <queue>
using namespace std;

class Node {
public:
    int data;
    Node* left;
    Node* right;

    Node(int value) {
        data = value;
        left = right = nullptr;
    }
};

void inorder(Node* root) {
    if (!root) return;
    inorder(root->left);
    cout << root->data << " ";
    inorder(root->right);
}

void preorder(Node* root) {
    if (!root) return;
    cout << root->data << " ";
    preorder(root->left);
    preorder(root->right);
}

void postorder(Node* root) {
    if (!root) return;
    postorder(root->left);
    postorder(root->right);
    cout << root->data << " ";
}
```

```
int main() {
    Node* root = new Node(10);
    root->left = new Node(20);
    root->right = new Node(30);
    root->left->left = new Node(40);

    cout << "Inorder: ";
    inorder(root);

    cout << "\nPreorder: ";
    preorder(root);

    cout << "\nPostorder: ";
    postorder(root);
}
```

Summary (Easy to Remember)

Operation	Description
Inorder	L → Root → R
Preorder	Root → L → R
Postorder	L → R → Root
Height	Longest path from root
Leaf Count	Nodes with no children
Level Order	BFS using queue
Search	Recursive left/right search

Binary Search Tree (BST)

A **Binary Search Tree (BST)** is a special type of binary tree where:

- Left child contains values **smaller** than the node
- Right child contains values **greater** than the node
- No duplicate values allowed

This property makes searching very fast.

BST Node Structure

```
class Node {
public:
    int data;
    Node* left;
    Node* right;

    Node(int value) {
        data = value;
        left = right = nullptr;
    }
};
```

Important Operations in BST

BST is mainly used for:

- Fast searching
- Fast insertion
- Fast deletion
- Sorted traversal

Let's see the operations one by one.

Insert into BST

Rule:

- If value < node → go left
- If value > node → go right
- Insert at the empty position

C++ Code:

```
Node* insertNode(Node* root, int value) {
    if (root == nullptr) {
        return new Node(value);
    }

    if (value < root->data)
        root->left = insertNode(root->left, value);
    else if (value > root->data)
        root->right = insertNode(root->right, value);

    return root;
}
```

Search in BST

Rule:

- If key == root → found
- If key < root → search in left subtree
- If key > root → search in right subtree

C++ Code:

```
bool searchBST(Node* root, int key) {
    if (root == nullptr) return false;

    if (root->data == key) return true;
    else if (key < root->data) return searchBST(root->left, key);
    else return searchBST(root->right, key);
}
```

Inorder Traversal (Sorted Output)

In BST, **inorder traversal gives sorted values**.

```
void inorder(Node* root) {
    if (root == nullptr) return;
    inorder(root->left);
    cout << root->data << " ";
    inorder(root->right);
}
```

Find Minimum Value (Leftmost Node)

```
int findMin(Node* root) {
    while (root->left != nullptr)
        root = root->left;
    return root->data;
}
```

Find Maximum Value (Rightmost Node)

```
int findMax(Node* root) {
    while (root->right != nullptr)
        root = root->right;
    return root->data;
}
```

Delete a Node from BST

Three cases:

1. **Leaf node** → delete directly
2. **One child** → replace node with child
3. **Two children** →
    - Find inorder successor (min value in right subtree)
    - Replace node's data
    - Delete that successor

C++ Code:

```
Node* deleteNode(Node* root, int key) {
    if (root == nullptr) return root;

    if (key < root->data)
        root->left = deleteNode(root->left, key);
    else if (key > root->data)
        root->right = deleteNode(root->right, key);
    else {
        // Case 1 & 2: Node with 0 or 1 child
        if (root->left == nullptr) {
            Node* temp = root->right;
            delete root;
            return temp;
        }
        else if (root->right == nullptr) {
            Node* temp = root->left;
            delete root;
            return temp;
        }

        // Case 3: Node with 2 children
```

```
        Node* temp = root->right;
        while (temp->left != nullptr) temp = temp->left;

        root->data = temp->data;
        root->right = deleteNode(root->right, temp->data);
    }
    return root;
}
```

Complete Example Program

```
#include <iostream>
using namespace std;

class Node {
public:
    int data;
    Node* left;
    Node* right;

    Node(int value) {
        data = value;
        left = right = nullptr;
    }
};

Node* insertNode(Node* root, int value) {
    if (!root) return new Node(value);

    if (value < root->data)
        root->left = insertNode(root->left, value);
    else if (value > root->data)
        root->right = insertNode(root->right, value);

    return root;
}

bool searchBST(Node* root, int key) {
    if (!root) return false;
    if (root->data == key) return true;
    else if (key < root->data) return searchBST(root->left, key);
    else return searchBST(root->right, key);
}

void inorder(Node* root) {
    if (!root) return;
    inorder(root->left);
    cout << root->data << " ";
    inorder(root->right);
}

int main() {
    Node* root = nullptr;

    root = insertNode(root, 50);
    insertNode(root, 30);
    insertNode(root, 70);
    insertNode(root, 20);
    insertNode(root, 40);
    insertNode(root, 60);
    insertNode(root, 80);
```

```
    cout << "Inorder (sorted): ";
    inorder(root);

    cout << "\nSearch 40: " << (searchBST(root, 40) ? "Found" : "Not
Found");
}
```

Summary Table

Operation	Time Complexity
Search	O(log n) average
Insert	O(log n) average
Delete	O(log n) average
Traversal	O(n)

What is an AVL Tree?

An **AVL Tree** is a **Self-Balancing Binary Search Tree (BST)**.

In an AVL Tree, for every node:

```
| Height(Left Subtree) − Height(Right Subtree) | ≤ 1
```

This difference is called the **Balance Factor (BF)**.

Balance Factor Formula

```
BF = Height(Left Subtree) − Height(Right Subtree)
```

Possible values:

- -1 → Balanced
- 0 → Balanced
- 1 → Balanced
- Less than -1 or greater than 1 → Unbalanced (Rotation required)

Why Do We Need AVL Tree?

In a normal BST:

Worst case time complexity = **O(n)**
(When tree becomes skewed like a linked list)

In AVL Tree:

Operation	Time Complexity
Search	O(log n)
Insert	O(log n)
Delete	O(log n)

Because the tree always remains balanced.

Node Structure in C++

```
struct Node {
    int data;
    Node* left;
    Node* right;
    int height;

    Node(int val) {
        data = val;
        left = right = NULL;
        height = 1;
    }
};
```

Most Used Functions in AVL Tree

1. Get Height

```
int getHeight(Node* node) {
    if (node == NULL)
        return 0;
    return node->height;
}
```

2. Get Balance Factor

```
int getBalance(Node* node) {
    if (node == NULL)
        return 0;
    return getHeight(node->left) - getHeight(node->right);
}
```

Rotations (Most Important Part)

There are 4 cases of imbalance:

Case	Situation	Rotation Needed
LL	Left-Left	Right Rotation
RR	Right-Right	Left Rotation
LR	Left-Right	Left + Right
RL	Right-Left	Right + Left

Right Rotation (LL Case)

```
Node* rightRotate(Node* y) {
    Node* x = y->left;
    Node* T2 = x->right;

    // Perform rotation
    x->right = y;
    y->left = T2;
```

```
    // Update heights
    y->height = 1 + max(getHeight(y->left), getHeight(y->right));
    x->height = 1 + max(getHeight(x->left), getHeight(x->right));

    return x;
}
```

Left Rotation (RR Case)

```
Node* leftRotate(Node* x) {
    Node* y = x->right;
    Node* T2 = y->left;

    // Perform rotation
    y->left = x;
    x->right = T2;

    // Update heights
    x->height = 1 + max(getHeight(x->left), getHeight(x->right));
    y->height = 1 + max(getHeight(y->left), getHeight(y->right));

    return y;
}
```

Insertion in AVL Tree

Steps:

1. Insert node like a normal BST.
2. Update height of ancestor nodes.
3. Calculate balance factor.
4. Perform rotation if unbalanced.

```
Node* insert(Node* node, int key) {

    // Step 1: Normal BST insertion
    if (node == NULL)
        return new Node(key);

    if (key < node->data)
        node->left = insert(node->left, key);
    else if (key > node->data)
        node->right = insert(node->right, key);
    else
        return node;  // No duplicates allowed

    // Step 2: Update height
    node->height = 1 + max(getHeight(node->left),
                           getHeight(node->right));

    // Step 3: Get balance factor
    int balance = getBalance(node);

    // Step 4: Rotation cases

    // LL Case
    if (balance > 1 && key < node->left->data)
        return rightRotate(node);
```

```
    // RR Case
    if (balance < -1 && key > node->right->data)
        return leftRotate(node);

    // LR Case
    if (balance > 1 && key > node->left->data) {
        node->left = leftRotate(node->left);
        return rightRotate(node);
    }

    // RL Case
    if (balance < -1 && key < node->right->data) {
        node->right = rightRotate(node->right);
        return leftRotate(node);
    }

    return node;
}
```

Inorder Traversal (To Verify BST)

```
void inorder(Node* root) {
    if (root != NULL) {
        inorder(root->left);
        cout << root->data << " ";
        inorder(root->right);
    }
}
```

Inorder traversal of a BST always prints values in sorted order.

Complete Example Program

```
#include <iostream>
using namespace std;

// Paste all functions here

int main() {
    Node* root = NULL;

    root = insert(root, 10);
    root = insert(root, 20);
    root = insert(root, 30);
    root = insert(root, 40);
    root = insert(root, 50);
    root = insert(root, 25);

    cout << "Inorder Traversal: ";
    inorder(root);

    return 0;
}
```

Output:

```
10 20 25 30 40 50
```

GRAPH REPRESENTATION

A **Graph** is a collection of:

- **Vertices (nodes)**
- **Edges (connections)**

Graphs can be:

- **Directed / Undirected**
- **Weighted / Unweighted**

To store a graph in memory, we mainly use:

1. Adjacency Matrix

An **Adjacency Matrix** is a **2D matrix** of size `V × V`
(where V = number of vertices).

- `matrix[i][j] = 1` → edge exists
- `matrix[i][j] = 0` → no edge

If weighted graph → store weight instead of 1.

Advantages

- Very fast edge lookup: **O(1)**
- Easy to implement

Disadvantages

- Uses a lot of memory: **O(V²)**
- Bad for sparse graphs (when edges are few)

C++ Code: Graph using Adjacency Matrix

```
#include <iostream>
using namespace std;

class GraphMatrix {
private:
    int vertices;
    int **adj;

public:
    GraphMatrix(int V) {
        vertices = V;

        adj = new int*[V];
        for (int i = 0; i < V; i++) {
            adj[i] = new int[V];
            for (int j = 0; j < V; j++)
                adj[i][j] = 0; // initialize with no edge
```

```
        }
    }

    // Add edge (undirected)
    void addEdge(int u, int v) {
        adj[u][v] = 1;
        adj[v][u] = 1;
    }

    void display() {
        cout << "Adjacency Matrix:\n";
        for (int i = 0; i < vertices; i++) {
            for (int j = 0; j < vertices; j++)
                cout << adj[i][j] << " ";
            cout << endl;
        }
    }
};
```

2. Adjacency List

Adjacency List stores graph as **list of lists**.

For each vertex, we store all connected neighbors.

Example for vertex **0 → [1,2]** means edges (0→1) and (0→2)

Advantages

- Very memory efficient: **O(V + E)**
- Best for sparse graphs
- Easy to iterate through neighbors

Disadvantages

- Checking edge existence takes O(degree)

C++ Code: Graph using Adjacency List

```
#include <iostream>
#include <vector>
using namespace std;

class GraphList {
private:
    int vertices;
    vector<vector<int>> adj;

public:
    GraphList(int V) {
        vertices = V;
        adj.resize(V);  // create V empty lists
    }

    // Add edge (undirected)
    void addEdge(int u, int v) {
        adj[u].push_back(v);
```

```
        adj[v].push_back(u);
    }

    void display() {
        cout << "Adjacency List:\n";
        for (int i = 0; i < vertices; i++) {
            cout << i << " -> ";
            for (int node : adj[i])
                cout << node << " ";
            cout << endl;
        }
    }
};
```

Example Usage (Main Function)

```
int main() {
    GraphMatrix gm(5);
    gm.addEdge(0, 1);
    gm.addEdge(1, 2);
    gm.addEdge(2, 3);
    gm.addEdge(3, 4);
    gm.display();

    cout << endl;

    GraphList gl(5);
    gl.addEdge(0, 1);
    gl.addEdge(1, 2);
    gl.addEdge(2, 3);
    gl.addEdge(3, 4);
    gl.display();

    return 0;
}
```

Comparison Table

Feature	Adjacency Matrix	Adjacency List
Space	$O(V^2)$	O(V + E)
Edge lookup	O(1)	O(degree)
Best for	Dense graphs	Sparse graphs
Easy to implement	Little Yes	Yes

BFS & DFS – Graph Traversal (NOTES + C++ Code)

1. BFS (Breadth-First Search)

BFS is a **level-order traversal** where you visit nodes **level by level** using a **queue**.

Key Points

- Uses **Queue (FIFO)**
- Explores neighbors first

- Good for shortest path in **unweighted** graphs
- Time Complexity: **O(V + E)**

Steps

1. Start from a source node
2. Push it into queue
3. Pop front element
4. Visit all unvisited neighbours
5. Repeat until queue is empty

2. DFS (Depth-First Search)

DFS explores **deeply first**, going down one branch before backtracking.

Key Points

- Uses **Stack** (or function recursion)
- Goes deep node → node → node
- Good for:
  - Detecting cycles
  - Topological sorting
  - Connected components
- Time Complexity: **O(V + E)**

Steps

1. Start from a node
2. Visit it
3. Move to unvisited neighbor
4. Repeat (recursive)
5. Backtrack

C++ Implementation: BFS + DFS using Adjacency List

```
#include <iostream>
#include <vector>
#include <queue>
using namespace std;

class Graph {
public:
    int vertices;
    vector<vector<int>> adj;

    Graph(int V) {
        vertices = V;
        adj.resize(V);
    }

    void addEdge(int u, int v) {
        adj[u].push_back(v);
        adj[v].push_back(u); // Undirected Graph
    }
```

```
    // BFS Traversal
    void BFS(int start) {
        vector<bool> visited(vertices, false);
        queue<int> q;

        visited[start] = true;
        q.push(start);

        cout << "BFS: ";

        while (!q.empty()) {
            int node = q.front();
            q.pop();
            cout << node << " ";

            for (int neigh : adj[node]) {
                if (!visited[neigh]) {
                    visited[neigh] = true;
                    q.push(neigh);
                }
            }
        }

        cout << endl;
    }

    // Helper function for DFS
    void DFSUtil(int node, vector<bool>& visited) {
        visited[node] = true;
        cout << node << " ";

        for (int neigh : adj[node]) {
            if (!visited[neigh])
                DFSUtil(neigh, visited);
        }
    }

    // DFS Traversal
    void DFS(int start) {
        vector<bool> visited(vertices, false);
        cout << "DFS: ";
        DFSUtil(start, visited);
        cout << endl;
    }
};

int main() {
    Graph g(6);

    g.addEdge(0, 1);
    g.addEdge(0, 2);
    g.addEdge(1, 3);
    g.addEdge(2, 4);
    g.addEdge(3, 5);

    g.BFS(0);
    g.DFS(0);

    return 0;
}
```

Example Output

```
BFS: 0 1 2 3 4 5
DFS: 0 1 3 5 2 4
```

Comparison Table

Feature	BFS	DFS
Data Structure	Queue	Stack / Recursion
Traversal	Level-wise	Deep-first
Shortest path (unweighted)	Yes	No
Memory usage	More	Less
Good for	Shortest path	Backtracking, Components